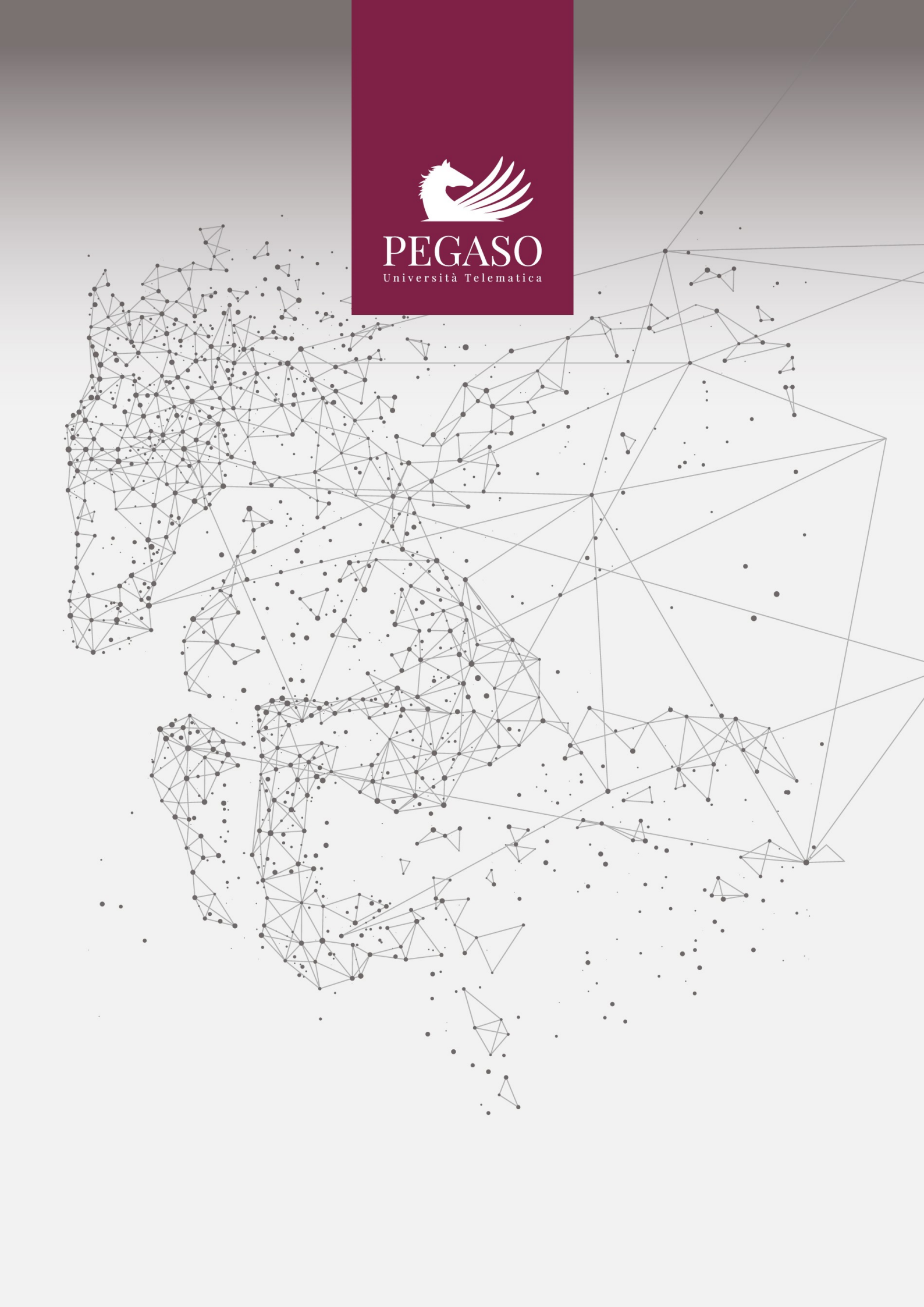

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. CONCETTI GENERALI E NOTAZIONE 3
2. OPERAZIONI E VINCOLI 8
3. ASSOCIAZIONI TRA CLASSI 11
BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Concetti generali e notazione

Una classe è l'espressione di un concetto (quindi con un confine definito) che rappresenta qualsiasi cosa materiale (es. orologio) oppure immateriale (es. organigramma aziendale). Più precisamente, una classe è un'astrazione di una categoria di entità (oggetti) dotate delle stesse caratteristiche e proprietà. E' l'evoluzione del tipo di dato astratto laddove si va ad incorporare sia lo stato (espresso tramite una serie di attributi) sia il comportamento (espresso tramite una serie di operazioni che agiscono in maniera esclusiva sui dati/attributi).

L'oggetto è una istanza di una classe, ovvero una manifestazione del concetto di classe in una realizzazione concreta (es. il mio orologio di una certa marca e con un certo colore). In accordo alla specifica della classe a cui appartiene, un oggetto ha un proprio stato e può rispondere a richieste per operazioni in base al proprio stato esibendo quindi un comportamento. Le operazioni rappresentano le computazioni o le azioni che gli oggetti appartenenti a quella classe sanno come effettuare. Possono riportare all'esterno informazioni sullo stato degli oggetti oppure modificare lo stato degli oggetti.

Il diagramma delle classi descrive il tipo degli oggetti (classi) che fanno parte di un sistema e le varie tipologie di relazioni statiche esistenti tra di essi. Il diagramma delle classi è una desrizione statica (ovvero strutturale) dei tipi di oggetti che compongono il sistema. Il diagramma delle classi mostra gli attributi (stato) e le operazioni di una classe e le restrizioni che si applicano al modo con cui gli oggetti sono relazionati tra loro.

È il modello più importante perché definisce gli elementi base del sistema. Ed è anche il più usato. Gli elementi di base dei diagrammi sono usati da tutti, quelli avanzati un po' meno ma senza dubbio sono molto usati anche perché si prestano ad essere usati sia da un punto di vista concettuale che da un punto di vista del software o progettazione.

Il diagramma delle classi può esprimere un modello di analisi (classi concettuali con pochi dettagli e funzioni/relazioni di base) o un modello di progetto (classi software con più dettagli) a seconda che siamo in fase di analisi e definizione dei requisiti oppure in fase di progettazione con la specifica (ad alto livello o con tutte le informazioni necessarie alla implementazione) delle entità software.

Il diagramma è la rappresentazione di un modello di analisi o di un progetto software basato su classe, relazione tra classi e specializzazione che si presuppone siano stati definiti secondo un approccio orientati agli oggetti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In UML una classe è rappresentata graficamente con un rettangolo avente 3 compartimenti. Il nome è l'unica proprietà obbligatoriamente presente: le altre proprietà (attributi e operazioni) sono incluse o omesse in funzione del grado di dettaglio che si vuole ottenere (Fig. 1).

In funzione delle esigenze di descrizione, è possibile rappresentare una classe con un semplice rettangolo contenente solo il nome, oppure mostrando anche le informazioni di dettaglio. Nella descrizione estesa, per ogni classe si specifica il nome che la identifica, gli attributi che la descrivono e le operazioni che si possono effettuare sulle sue istanze. Il nome della classe è scritto con la lettera maiuscola ed è autodescrittivo. La differenza tra la prima e la seconda rappresentazione è che nel primo a sinistra indica che il modello non precisa nulla riguardo agli attributi e alle operazioni, mentre nel secondo caso si vuole indicare che nel modellare sono state prese in considerazione le informazioni sugli attributi e sulle operazioni ma si è deciso che la classe non deve avere né attributi e né operazioni. Nel terzo caso si è specificato sia gli attributi che le operazioni.

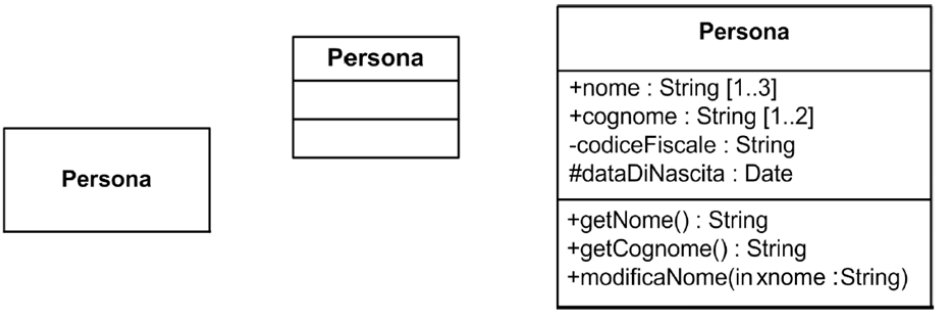


Fig. 1: Notazione della classe

Vediamo quindi più nel dettaglio questi aspetti.

Un **attributo** descrive una caratteristica propria di ogni istanza della classe (Fig. 2).

Visibilità: indica se l'attributo può essere visibile ad altre classi o meno. I diversi tipi di visibilità sono:

■ *pubblico*: indicato con il simbolo "+" accanto al nome dell'attributo; indica che l'attributo considerato è visibile all'esterno da tutte le classi;

■ *protetto*: indicato con il simbolo "#" accanto al nome dell'attributo; indica che l'attributo di una classe è visibile solo dalle classi che ereditano proprio da quest'ultima;

■ *privato*: indicato con il simbolo "–" accanto al nome dell'attributo; indica che l'attributo è visibile solo dalla classe in cui è definito.

Se omessa di default l'attributo è privato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nome: identifica l'attributo. I nomi devono essere dati in modo che descrivano il significato dell'attributo stesso.

Molteplicità: indica il numero di istanze dell'attributo che una classe può avere. Ad esempio, gli attributi nome e cognome hanno rispettivamente molteplicità da 1 a 3 e da 1 a 2: una persona può avere fino a 3 nomi e fino a 2 cognomi. Se omessa il valore di default è 1.

■ *Tipo*: indica il tipo di riferimento dell'attributo. Un attributo può essere, ad esempio, di tipo Integer, String, Boolean, o anche un'istanza di un'altra classe.

■ *Valore di default*: indica il valore iniziale da assegnare a un attributo.

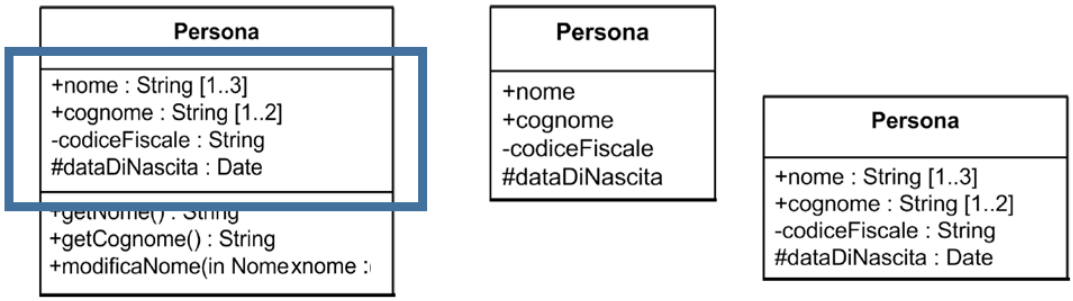


Fig. 2: Attributi di una classe

Le **operazioni** rappresentano le computazioni o le azioni che gli oggetti appartenenti a una classe sanno come effettuare. Gli oggetti interagiscono tra di loro richiedendo reciprocamente servizi o informazioni attraverso invocazione delle operazioni che rendono visibili all'esterno (secondo la definizione della classe). Alcune di queste operazioni riportano all'esterno le informazioni sullo stato interno degli oggetti. Altre operazioni sono invece invocabili solo dall'oggetto stesso e generalmente modificano lo stato interno dell'oggetto.

Un'*operazione* è descritta dalle seguenti informazioni (Fig. 3).

■ *Visibilità*: indica se l'operazione è visibile da altre classi o meno. Il concetto e i tipi di visibilità possibili sono gli stessi utilizzati per la visibilità degli attributi.

■ *Nome*: identifica l'operazione.

■ *Parametri dell'operazione*: indicano i parametri di ingresso. Per ogni parametro si indica:

■ *direzione*: indica se il parametro è di ingresso (in), di uscita (out) o di ingresso uscita (inout); nel caso non si specifichi nulla il parametro è definito di ingresso (in);

■ *nome*: identifica e descrive il parametro;

■ *tipo*: indica di che tipo deve essere il parametro fornito (Integer, String, Boolean…);

■ *valore di default*: indica un valore iniziale predefinito da assegnare al parametro.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

■ *Tipo di ritorno*: indica il tipo del risultato che viene restituito dall'operazione. Si ricorda che un'operazione può non restituire alcun valore: in questi casi non si indica nulla o si utilizza la parola chiave void.

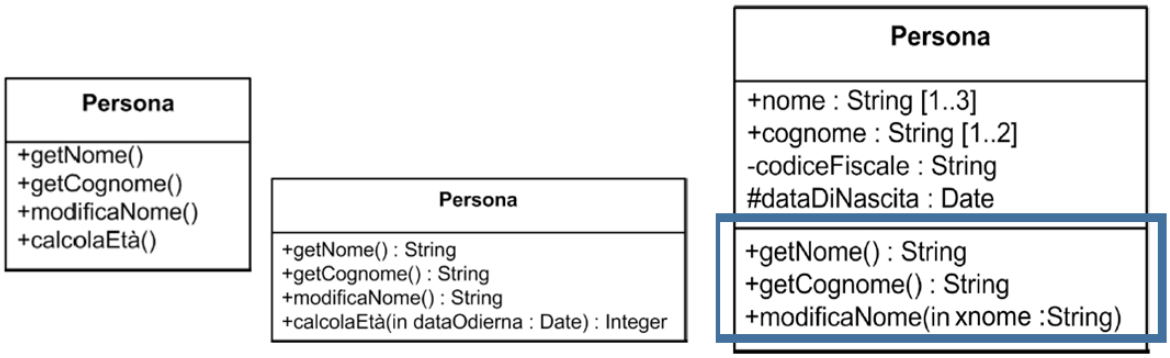


Fig. 3: Operazioni di una classe

Come si può vedere dalla Fig. 2 e dalla Fig. 3, il nome dell'operazione, come quello degli attributi, è obbligatorio, mentre tutto il resto della descrizione può essere omesso, in funzione del grado di dettaglio che si vuole dare allo schema.

Come viene rappresentato un oggetto?

Un oggetto viene rappresentato da un rettangolo come le classi, con la differenza che il nome dell'oggetto e della classe sono sottolineati per indicare che si tratta di un'istanza. Il rettangolo contiene i nomi dell'oggetto e della classe d'appartenenza, sottolineati e separati dal carattere ':', ed opzionalmente gli attributi con i rispettivi valori. Il nome della classe o quello dell'oggetto possono mancare. Più precisamente, il formato per indicare il nome dell'oggetto è nome-istanza : nome-classe (può comparire o solo nome-istanza oppure solo : nome-classe). Per indicare eventualmente gli attributi il formato è *nome-attributo* = «*valore*» con l'elenco degli attributi non necessariamente esaustivo.

In UML la rappresentazione esplicita di oggetti è piuttosto rara, in quanto il lavoro di analisi, specifica e progettazione si basa essenzialmente sulle classi e le loro relazioni, però gli oggetti possono essere usati per esemplificare delle situazioni particolari o tipiche. La Fig. 4 mostra la rappresentazione di una classe e di una sua istanza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

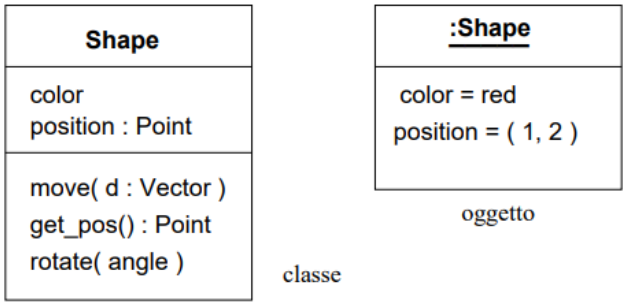


Fig. 4: Rappresentazione di un oggetto (a destra) relativo ad una classe (a sinistra)

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni e vincoli

Detto in altri termini anche più intuitivi, gli attributi individuano lo stato interno degli oggetti di una data classe mentre le operazioni rappresentano le computazioni che una classe sa come effettuare. Possono modificare lo stato degli oggetti oppure possono riportare all'esterno informazioni sullo stato degli oggetti. Per le operazioni pubbliche queste rappresentano la modalità con cui gli oggetti possono interoperare o mandarsi messaggi modificando reciprocamente lo stato (in genere rappresentato da più attributi). Per gli attributi privati (anche se non necessariamente) possono essere previste delle operazioni che forniscono i valori all'esterno.

E' utile osservare la distinzione fra operazione e metodo. In UML vi è una netta distinzione fra metodo e operazione: un'operazione è la specifica di una computazione, specifica che si può ridurre alla semplice descrizione dei parametri o includere vincoli (per esempio, precondizioni, invarianti e postcondizioni) e varie annotazioni. Un metodo è l'implementazione in un linguaggio di programmazione.

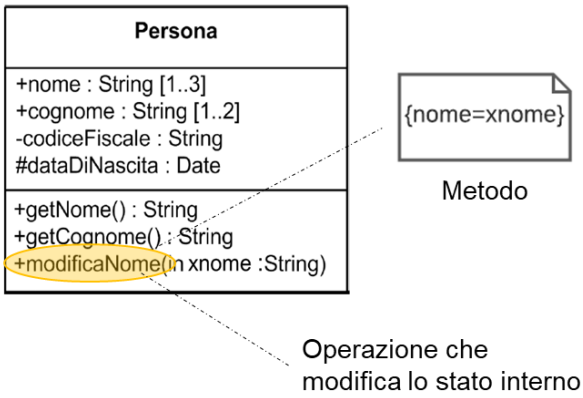


Fig. 5: Rappresentazione di un oggetto (a destra) relativo ad una classe (a sinistra)

Possiamo vedere operazioni che riportare all'esterno informazioni sullo stato degli oggetti (get) oppure per modificare lo stato degli oggetto rappresentato da uno o più attributi (set). E' necessario inserire questi metodi per mediare l'ottenimento e la modifica di attributi che sono privati o protetti. anche qualora questi siano pubblici e quindi visibili anche all'esterno. Per gli attributi pubblici quest'ultima possibilità è superflua visto che sono visibile direttamente all'esterno, tuttavia più essere comunque consigliabile mettere a disposizione queste operazioni per mediare l'ottenimento di attributi anche se pubblici. In Fig. 5 è indicata l'implementazione, ovvero il metodo, dell'operazione modificaNome.

L'UML non ha elementi di modello destinati a descrivere i metodi, che comunque non interessano in fase di specifica. Se bisogna descrivere l'implementazione di un'operazione, per esempio nella fase di

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

codifica, si può usare la notazione pensata per aggiungere note/commenti e associare tale nota/commento all'operazione.

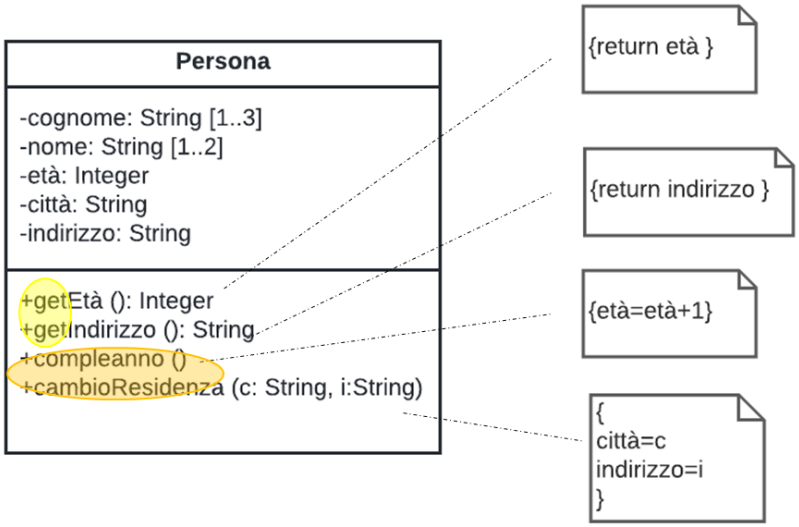


Fig. 6: Rappresentazione di un oggetto (a destra) relativo ad una classe (a sinistra)

Questa è invece una diversa versione della classe per rappresentare il concetto di Persona. Tutti gli attributi sono privati e i metodi di get e set riguardano attributi legati all'età e alla residenza (potrebbe trattarsi ad esempio di un programma che è incentrato sulla demografia ovvero su studio quantitativo dei fenomeni riguardanti lo stato e il movimento della popolazione). Si vuole infatti che le persone devono essere in grado di compiere gli anni e di cambiare residenza. Quindi oltre ai metodi get per ottenere età e residenza si forniscono metodi set per due operazioni che riguardano il far compiere gli anni e un'altra per aggiornare la residenza.

Se si vogliono documentare dei vincoli, il diagramma delle classi è un ottimo posto per farlo. I vincoli riguardano condizioni imposte sulla semantica di un elemento del diagramma quale ad esempio una classe o un attributo. Ad esempio può essere specificare il valore iniziale di un attributo, un vincolo di valori tra attributi diversi della stessa classe oppure tra classi diverse. L'importante è che non siano inconsistenti tra loro (ad esempio si chiede che un attributo di tipo numerico assume sempre valori maggiorni di 5 e minori di 2) per rendere il sistema rappresentato dal modello realizzabile.

I vincoli possono essere inclusi in note tra parentesi graffe ed espressi in molti modi: attraverso linguaggio naturale, un linguaggio formale (linguaggio di programmazione, espressione matematica o

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

booleana), Object Constraint Language (OCL). OCL è basato sul calcolo dei predicati e essendo un formalismo si presta meno ad interpretazioni (purché chi lo uso lo conosce bene).

Nell'esempio in Fig. 7 il vincolo esprime che ogni impiegato, se ha un capo, lavora per la stessa compagnia del suo capo.

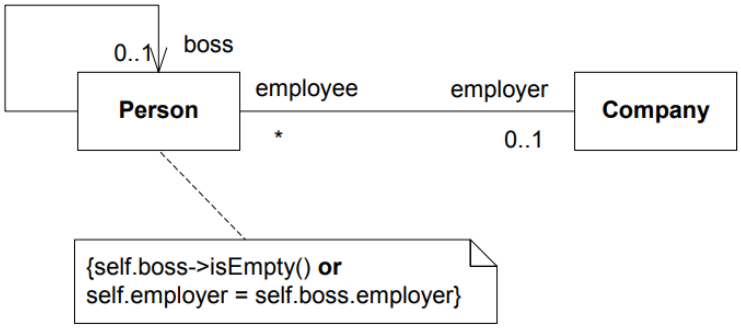


Fig. 6: Esempio di vincolo su una classe

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Associazioni tra classi

Un'associazione tra due classi indica che gli oggetti di una classe posta ad un estremo dell'associazione «riconosce» gli oggetti all'altro estremo, ovvero ne è a conoscenza, li può raggiungere e mandare loro dei messaggi. Un'associazione può coinvolgere più di due classi e la stessa classe più di una volta. Tra le relazioni è anche la più flessibile e la meno vincolante: stabilisce una semplice relazione di riconoscimento tra classi (la classe ha un riferimento della classe alla quale è associata e quindi sa che esiste e come "raggiungerla".

La notazione, nel caso generico e minimale, è una semplice linea che collega le classi coinvolte.

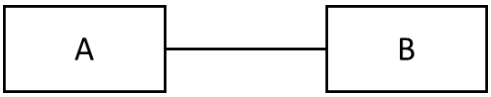


Fig. 5: Associazione tra classi

Navigabilità: Le associazioni possono essere o meno orientate e di conseguenza ci può essere una direzione di navigabilità. La navigabilità è rappresentata con una freccia aggiunta all'associazione. La navigazione indica la direzione di percorrenza di un'associazione: in particolare, in presenza di quest'informazione si deduce che la classe sorgente ha un riferimento alla classe obiettivo della navigazione. La navigabilità è opzionale e, quindi, può anche non essere inserita in un diagramma.

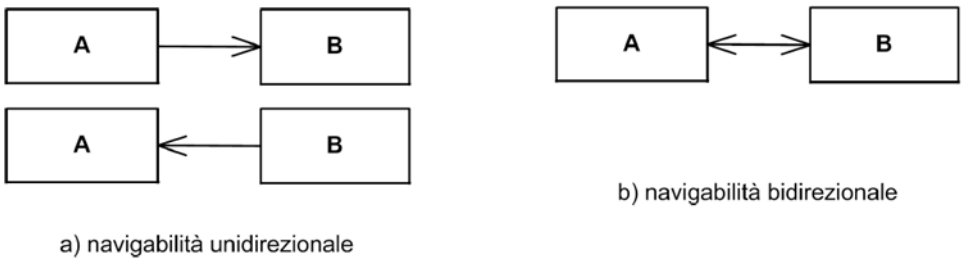


Fig. 6: Navigabilità di un'associazione

I tipi di navigabilità sono i seguenti.

■ *Unidirezionale*: indica che è specificata una particolare direzione dell'associazione, mentre l'altra non è specificata e rimane, quindi, indefinita. Negli esempi sono mostrate le due alternative unidirezionali: classe sorgente ha un riferimento alla classe obiettivo della navigazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

■ *Bidirezionale*: la navigabilità è specificata in entrambe le direzioni. Di conseguenza, l'associazione presenta la punta di una freccia in entrambe le estremità, come mostrato nell'esempio: le classi hanno un riferimento reciproco l'una dell'altra.

UML permette l'uso della linea semplice non solo quando non si vuole mostrare una navigabilità particolare ma anche per indicare un'associazione bidirezionale, anche se la linea con la doppia freccia risulta più chiara e toglie da questa ambiguità.

In genere l'associazione fra classi avviene per presenza di attributi della classe associata (tipicamente non di tipi elementari).

Un' associazione è definita in modo univoco tramite un **nome** che descrive il significato dell'associazione stessa ed è posto come etichetta al centro. Il nome è spesso indicato tramite un verbo in questo modo la relazione può essere usata così com'è per dare origine a frasi di senso compiuto.

Quando bisogna leggere un nome tenendo conto di un verso privilegiato di lettura dell'associazione, si può indicare mediante un triangolo nero alla fine del nome. Il triangolo nero posto di fianco al nome dell'associazione indica come leggere e interpretare il nome dell'associazione stessa ("una persona *possiede* un'auto" e non viceversa).

Un'associazione è descritta, inoltre, dal **ruolo** ricoperto dalle classi coinvolte. Il ruolo ricoperto da una classe in un'associazione è rappresentato accanto a essa. Per esempio, nella Fig. 6 le parole chiave «Possiede» e «Appartiene a» identificano i ruoli delle classi Persona e Auto nella relazione «possiede».

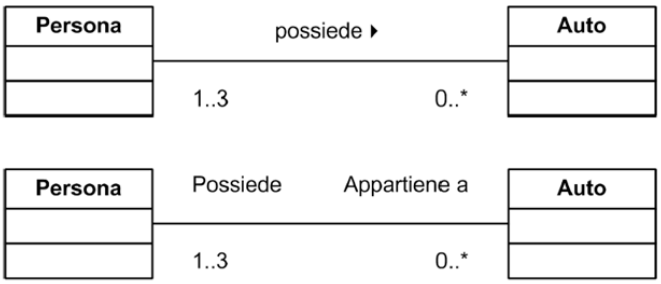


Fig. 6: Nome e ruoli delle classi in un'associazione e molteplicità

Sempre in riferimento alla Fig. 6, una caratteristica importante delle associazioni è il concetto di molteplicità. La *molteplicità* di un'associazione è riferita alle classi: indica il numero di istanze di una determinata classe coinvolte nella relazione. Le dichiarazioni di molteplicità possibili sono:

■ 1: indica che nell'associazione è coinvolta una e una sola istanza della classe

■ 0..1: indica che nell'associazione può essere coinvolta o meno un'istanza di una classe

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

■ 0..*/1..*: indica che le istanze coinvolte possono essere molte. In questo caso non è indicato un numero massimo preciso; la presenza del limite inferiore uguale a uno indica che almeno un'istanza della classe è coinvolta

■ 0..n/1..n: indica che le istanze coinvolte possono essere al massimo n

■ n: indica che le istanze della classe considerata coinvolte nell'associazione sono esattamente n.

La molteplicità di un'associazione è indicata come illustrato nel diagramma della Fig.6 che descrive la relazione di proprietà che lega una persona a un'auto. I modi per leggere tale relazione sono due.

■ Una persona possiede zero o più auto. In questo caso la molteplicità "0" indica che una persona può anche non possedere alcuna auto.

■ Un'auto appartiene a un numero di persone che varia tra uno e tre.

E' possibile realizzare anche un'*associazione riflessiva*. Un'associazione è riflessiva se le istanze tra cui è applicata appartengono alla stessa classe. Nella Fig. 7 si descrive che una directory può essere contenuta in un'altra e può a sua volta contenere zero o più directory.

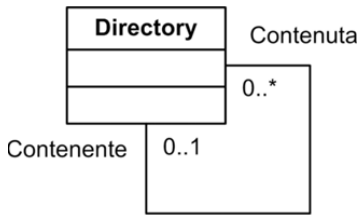


Fig. 7: Associazione riflessiva

I ruoli e i nomi aiutano a leggere e interpretare il diagramma guidando percorsi semantici che a partire da una istanza di una delle classi dell'associazione producono come risultato insieme di istanze di un'altra classe nel diagramma a cui sono relazionate

Nella Fig. 6 le parole chiave Possiede e Appartiene a identificano i ruoli delle classi Persona e Auto nella relazione «possiede». I ruoli «Possiede»/ «Appartiene a» o il nome «possiede» (abbinato ad una freccia come nell'esempio) aiutano a leggere il diagramma e a interpretare l'associazione rappresentando dei percorsi semantici che a partire da una istanza di una delle classi del diagramma producono come risultato insieme di istanze di un'altra classe nel diagramma a cui sono relazionate (e il tipo di associazione). Ad esempio se John è un'istanza della classe Persona, l'espressione john.possiede rappresenta l'informazione sull'insieme di tutte le istanze della classe Auto che sono in relazione con John e quindi che lui possiede. Questa associazione si può anche estrinsecare nel codice tramite un attributo, ad es. automobili di tipo Auto, che avrà tante istanze quante sono le auto possedute da John che, secondo quanto indicato dalla molteplicità 0..*, possono essere 0 oppure svariate senza un numero massimo. Lo stesso vale

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

partendo da Auto andando così a ritrovare le istanze della classe Persona ad essa collegata. Così partendo da una particolare Fiat Punto è possibile trovare tutte le persone (ovvero istanze della classe Persona) a cui appartiene tale auto che, secondo quanto indicato dalle molteplicità 1..3, possono essere da 1 a 3 (ovvero un auto deve almeno avere un proprietario).

Va da sé che seguendo il valore di tali attributi, ad esempio partendo da una particolare Fiat Punto (istanza della classe Auto) e andando a trovare il proprietario e poi si guarda la collezione di auto possedute da quella persona tale insieme dovrebbe contenere la Fiat Punto.

Si noti che il concetto di navigabilità è diverso dal verso di lettura dell'associazione (il triangolo nero della Fig. 6). Il verso di lettura serve solo a facilitare la comprensione del diagramma e a interpretare correttamente il nome dell'associazione: Persona e Auto sono in relazione perché una persona può possedere un'auto.

La navigabilità (la freccia della linea che collega le due classi) chiarisce in quale direzione da un'istanza di una classe si può passare a istanze di altre classi seguendo una specifica direzione. Un oggetto di tipo Persona sa se e quali oggetti di tipo Auto possiede.

Anche un'associazione può essere specificata da particolari vincoli. In questo caso i *vincoli* sono delle condizioni espresse sull'associazione stessa o sulle classi coinvolte: tali condizioni devono essere verificate per realizzare l'associazione.

Nell'esempio mostrati si ha dei vincoli sulle classi coinvolte indicate con espressioni matematiche agli estremi dell'associazione: il vincolo sull'associazione acquista indica che solo una persona che abbia compiuto diciotto anni può comprare un'auto, mentre il vincolo dell'associazione prende in affitto indica che solo chi è munito di patente può affittare un'auto (età e haPatente sono due ulteriori attributi di Persona).

In UML è possibile anche vincolare tra loro due associazioni: nell'esempio mostrato il vincolo {OR} descrive che una persona può affittare un'auto oppure comprarne una, ma non entrambe le cose.

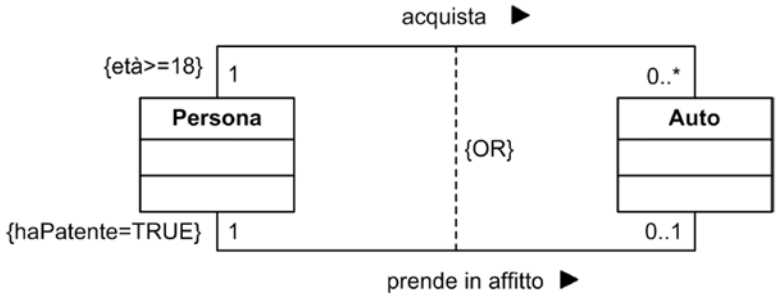


Fig. 8: Vincoli delle associazioni

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- A. Cockburn, Writing Effective Use Cases, 2001, Reading, M.A.: Addison-Wesley
- M. Fowler, UML Distilled, Pearson, Quarta edizione
- R. S. Pressman, Principi di Ingegneria del software, McGraw-Hill, Quinta edizione
- C. Larman, Applicare UML e i pattern, Pearson, Quinta edizione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).